

FLDSMDFR (Graficadora en Assembler)

Manual Técnico

- Requisitos:

Se deben contar con los siguientes programas instalados en la computadora:

- DOS-BOX 0.74-3
- Código fuente/ejecutable de la aplicación colocado en la carpeta a montar en DOS-BOX
- Compilador NASM (recomendado: versión 16-bit para compilar dentro de DOS-BOX.
Opcional: versión 32/64 bit para correr dentro del sistema operativo host.)
- ALINK v1.6 para OBJ y LIB, que genera archivos COM/EXE
- Opcional: AFD-Pro (Advanced Full-screen Debugger Professional)
- Opcional: EDIT.EXE (Editor de texto en DOS)

- Procedimiento de inicio de DOS-BOX

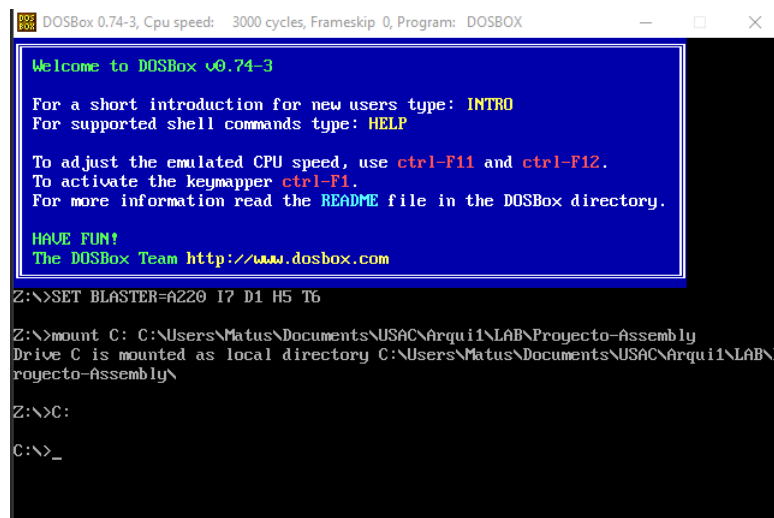
Luego de haber instalado DOS-BOX y colocar el ejecutable en una carpeta (preferiblemente lo más cercano a la raíz del disco donde se almacena. Se procede a abrir DOS-BOX y ejecutar los siguientes comandos:

```
mount C: C:\dir
```

donde C:\dir es el directorio donde se encuentran los requisitos descritos (Compilador, linker, código fuente y debugger). Este puede ser en otro disco. Pero el disco a montar si debe ser el C:

Luego, navegamos al lugar recién montado:

```
C:
```



Una vez hecho esto, podemos proceder al proceso de compilado.

- **Procedimiento de compilado de la aplicación**

Se ejecuta el siguiente comando para compilar el programa:

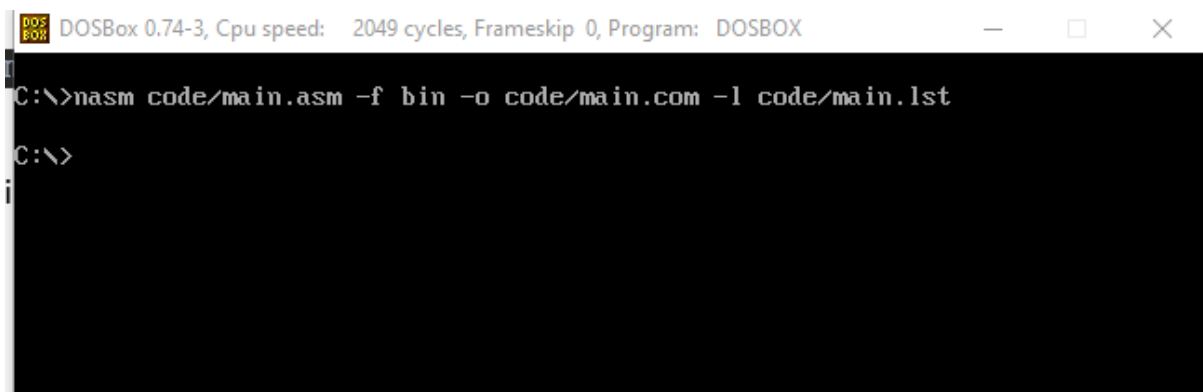
```
nasm code/main.asm -f bin -o code/main.com -l code/main.lst
```

-f especifica el tipo de aplicación a generar. Entre las opciones se encuentran:

- bin: archivo sin header (flat)
- elf: Objeto ELF
- aout: a.out
- coff: Objeto COFF (Microsoft Windows)
- macho: Objeto Mach-O (MAC OSX)
- rdf: Objeto RDOFF
- win32: Objeto de 32-bit de Windows
- win64: Objeto de 64-bit de Windows

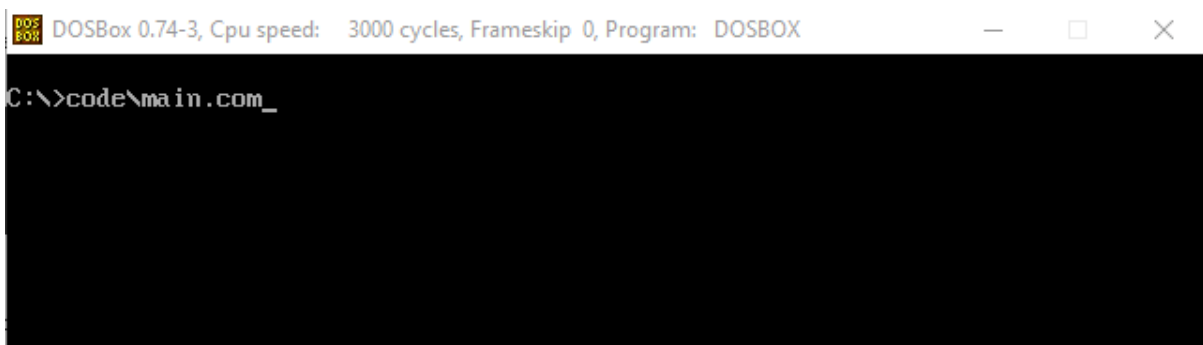
-o especifica el archivo de salida, en este caso almacenado en code/main.com

-l genera un archivo de listado. Este archivo contiene el código fuente, direcciones (relativas) del código maquina y etiquetas y símbolos que se encuentran en el mismo.



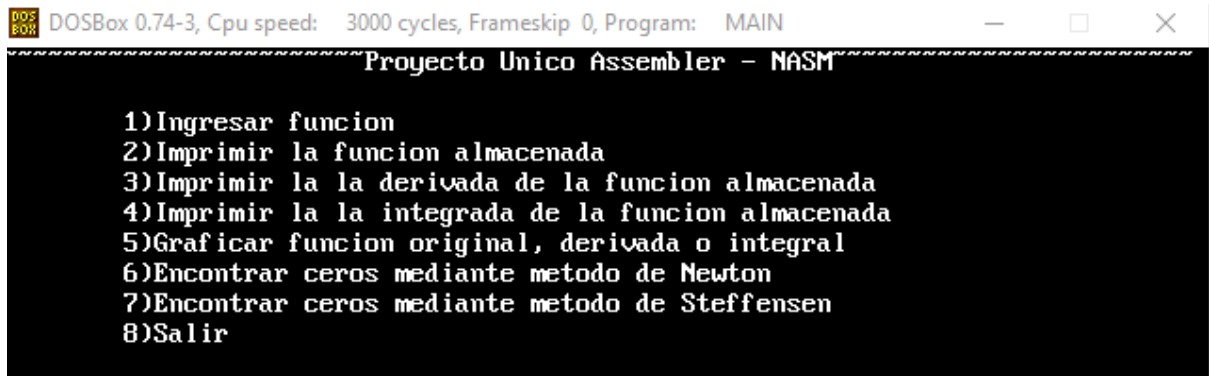
A screenshot of a DOSBox terminal window. The title bar reads "DOSBox 0.74-3, Cpu speed: 2049 cycles, Frameskip 0, Program: DOSBOX". The terminal shows the command `C:\>nasm code/main.asm -f bin -o code/main.com -l code/main.lst` being entered and executed. The prompt `C:\>` is visible on the next line.

Una vez compilado, quedan generados los archivos .com y .lst. El que realmente interesa para correr la aplicación es el .com. para correr la aplicación, basta con ingresar su nombre (y ubicación) como comando.



A screenshot of a DOSBox terminal window. The title bar reads "DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX". The terminal shows the command `C:\>code\main.com` being entered. The cursor is at the end of the command.

Nota: Con CTRL+F11/CTR+F12 se puede decrementar/incrementar la velocidad emulada de CPU. Aumentándola, se puede ver reducido el tiempo de compilación del código fuente.

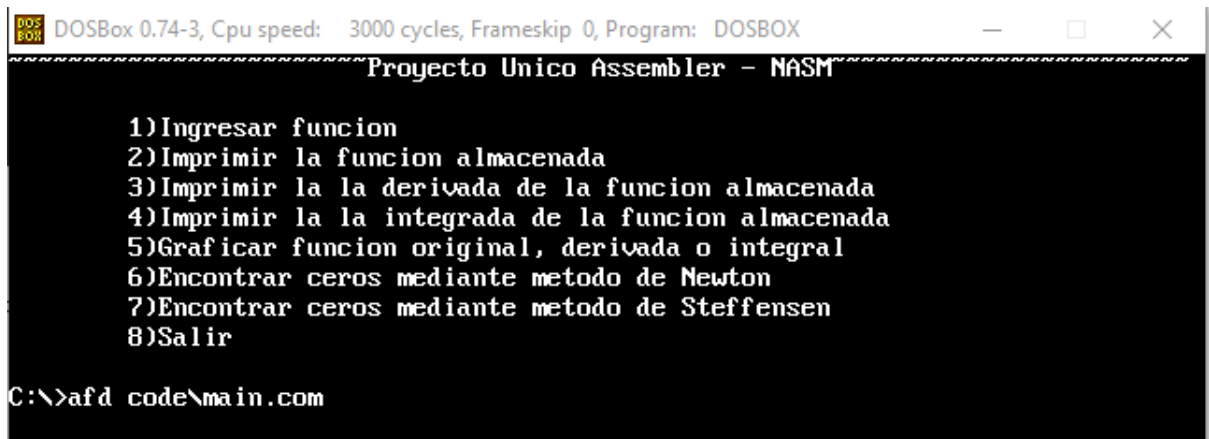


DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: MAIN

Projecto Unico Assembler - NASM

- 1)Ingresar funcion
- 2)Imprimir la funcion almacenada
- 3)Imprimir la la derivada de la funcion almacenada
- 4)Imprimir la la integrada de la funcion almacenada
- 5)Graficar funcion original, derivada o integral
- 6)Encontrar ceros mediante metodo de Newton
- 7)Encontrar ceros mediante metodo de Steffensen
- 8)Salir

Si se desea debuggear la aplicación en vez de solamente correrla, podemos hacer uso de AFD.EXE, brindándole como argumento la ubicación del archivo .com



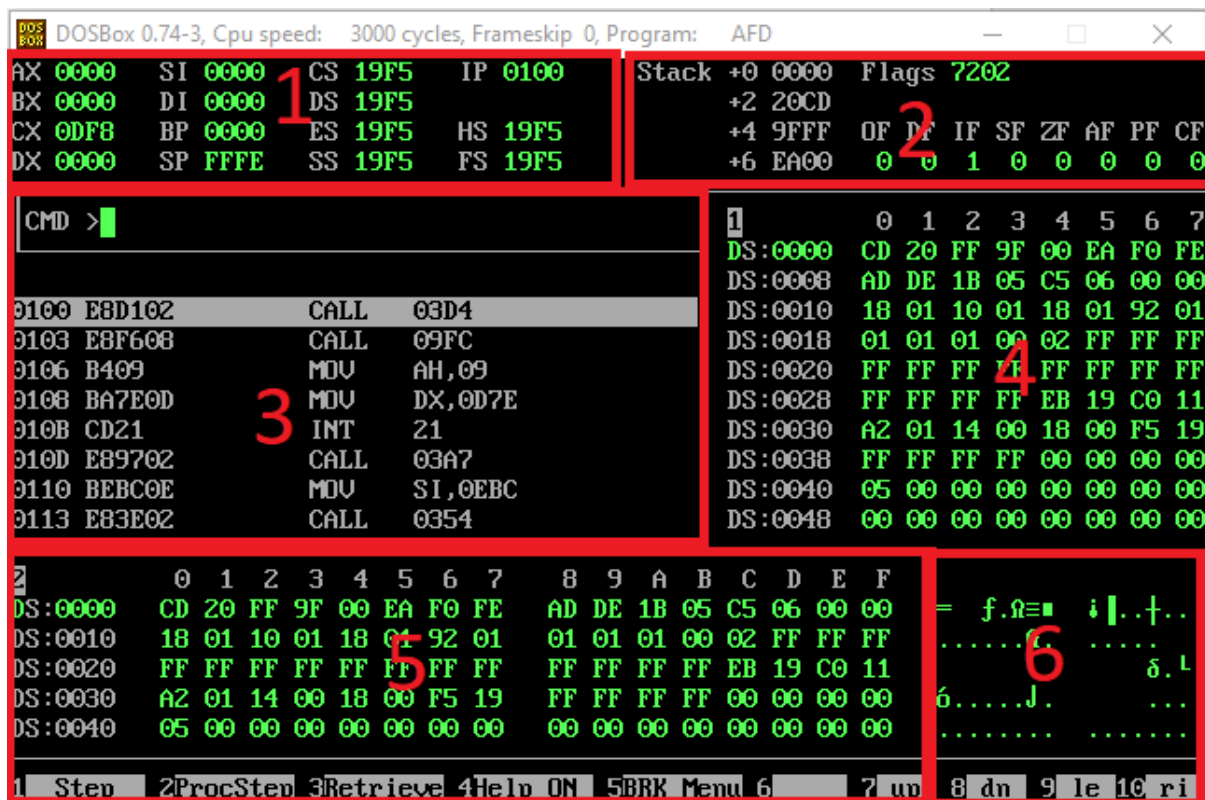
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX

Projecto Unico Assembler - NASM

- 1)Ingresar funcion
- 2)Imprimir la funcion almacenada
- 3)Imprimir la la derivada de la funcion almacenada
- 4)Imprimir la la integrada de la funcion almacenada
- 5)Graficar funcion original, derivada o integral
- 6)Encontrar ceros mediante metodo de Newton
- 7)Encontrar ceros mediante metodo de Steffensen
- 8)Salir

C:\>afd code\main.com

Esto nos abrirá el debugger el cual consta de las siguientes ventanas principales:



1. Registros: El estado actual de los valores de registros. Se pueden cambiar usando el comando REG=VAL (ej.: AH=0026)
2. STACK + Banderas: Estado actual de los valores de banderas, se pueden cambiar usando el comando FLAG=VAL (ej.: CF=1, o todas las banderas a la vez usando FL=XXXX)
3. Instrucciones del programa: Se pueden ver todas las instrucciones programadas en el código fuente. Acá se ven los comandos ingresados del debugger. La lista de comandos completos se puede ver usando F4
4. Vista de Memoria 1: vista general y alargada de memoria. Se puede navegar usando el comando M1 ADDR (ej.: M1 65B). La memoria se puede modificar directamente con el comando F addr ,repeat_count, string; llenando la memoria con el string dado x cantidad de veces.
5. Vista de memoria 2: Vista alineada y más específica de la memoria. Se puede navegar usando el comando M2 ADDR (ej.: M1 65B)
6. Representación ASCII: Representación ASCII (si es visible, si no un ".") del código hexagesimal almacenado, alineado con la vista de *Vista de memoria 2*

Se puede salir del debugger usando el comando *Quit*